Towards Decentralized Model Persistence

Research proposal for a study of content-addressed storage in model-driven engineering

Alfa Yohannis
Department of Informatics, Pradita University
Kabupaten Tangerang, Banten, Indonesia
alfa.ryano@pradita.ac.id

September 2026

Duration	6 months
Target venue	Journal of Systems and Software (JSS), with an earlier conference paper on the two of the research questions

1 Summary

Model-driven engineering (MDE) stores models in files and in repositories, and a model refers to another model through an address. In the Eclipse Modeling Framework (EMF), that address is a Uniform Resource Identifier (URI) such as `file://` or `platform://`, which names a location. The content at a location can change while the address stays the same, so a reference resolves to whatever the location holds at the moment of the read. The InterPlanetary File System (IPFS) inverts that relation, because the address is a content identifier (CID) computed from the content itself. A change to a model therefore produces a new address, and an existing reference keeps pointing at the version that was referenced.

The study will build a persistence layer that stores EMF models on IPFS, will work out how references between resources survive when identifiers change on every modification, and will measure the cost. The outcome is a working artifact, an evaluation on generated models and on large reverse-engineered models, and a journal paper.

2 Background

EMF holds a model in a `Resource`, groups resources in a `ResourceSet`, and resolves a reference into another resource through a proxy [19]. Serialization uses the XML Metadata Interchange (XMI) format, where a cross-resource reference is a URI with a fragment, for example `href="model.xmi#//Person"` [15]. Research on model persistence has mainly followed scalability [12]. Morsa stores large models in a non-relational database and loads parts on demand [16]. NeoEMF offers several database back ends behind one framework [6]. The Connected Data Objects (CDO) repository stores and versions models on a central server [4]. Hawk indexes large collections of models held in version control systems [2].

Content addressing comes from a different tradition. Venti used a hash of the block content as the identifier of a block in archival storage [17]. Git stores every object under the checksum of the object content [5]. Nix derives component paths from cryptographic hashes, so several versions of a component coexist [8]. Software Heritage preserves source code under intrinsic identifiers [7]. IPFS combines content addressing with a peer-to-peer network [1, 20], and the InterPlanetary Name System (IPNS) adds mutable names on top of fixed identifiers.

3 Problem statement

A location-based address carries no information about the version behind the address. Three settings show the consequence.

First, several models in a team often reference one metamodel or one shared library model. An edit to the shared resource changes the meaning of every referring model, and no referring model records which version was intended.

Second, reverse engineering produces models of a code base. A later analysis should run on the same snapshot as an earlier analysis, yet a file path does not name a snapshot.

Third, organizations exchange models across a boundary. A path inside one organization has no meaning inside another organization, so the parties need agreed names or a shared server.

Figure 1 contrasts a local development setting for a model that is split into fragments with the decentralized setting that the study targets. In the local setting, every party reads and writes one shared location, and each reference names a path. In the decentralized setting, every fragment becomes an immutable block, each reference names the CID of the target, and a mutable IPNS name marks the current version.

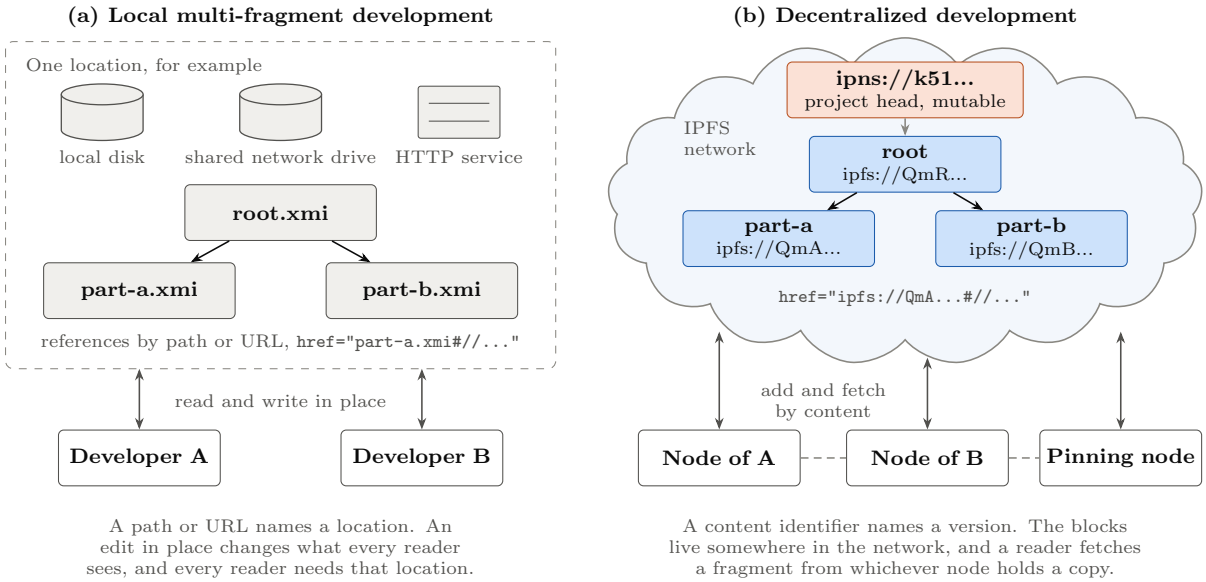


Figure 1: Development of a model that is split into fragments. (a) In a local setting, the fragments sit in one shared location, and each reference names a path. (b) In the decentralized setting of the study, each fragment is an immutable block on IPFS, each reference names a CID, and a mutable IPNS name marks the current version of the project.

Figure 2 contrasts the two kinds of address. A reference that names a version would remove the ambiguity in all three settings. Content addressing offers such a reference, yet the model persistence frameworks of MDE assume a stable address with changing content. The assumption runs through the resource, the factory, the URI handler and the serialization of references, so the transfer is not a configuration change.

4 State of the art and the gap

Three lines of work touch the problem without closing the problem.

Scalable persistence keeps identifiers stable inside a store, because the store owns the identity of an element [16, 6, 4]. Versioning systems such as EMFStore record operations between versions [11], and comparison tools such as EMF Compare compute differences between two states [9]. Both lines answer the question of what changed, while neither gives an immutable address that a third party can quote.

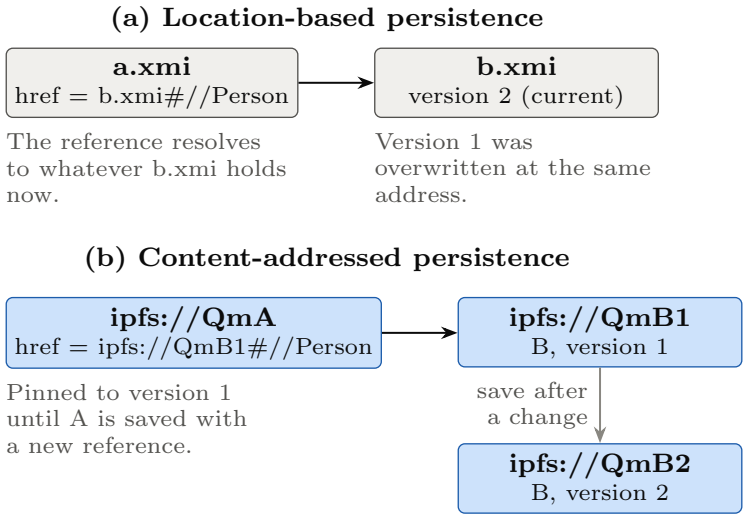


Figure 2: Two kinds of address. (a) A file path resolves to the current content at the path. (b) A content identifier resolves to one version, so a reference keeps pointing at the version that was referenced.

Content-addressed storage outside MDE gives exactly such an address, and package managers already use the idea for dependencies. A lock file records an exact dependency tree, Go modules record checksums, and reproducible builds check that a binary matches the source code [14, 13]. None of these applies content addressing to the resources and the references of a modeling framework.

The gap is therefore concrete. No reviewed approach stores EMF models under content-addressed identifiers, and no approach describes how cross-resource references behave when every save produces a new identifier.

5 Objectives and research questions

The study will answer three questions.

RQ1. Architecture. What are the architectural challenges of extending a model persistence framework to support content-addressed decentralized storage, and how can the challenges be addressed? Sub-questions cover the mapping of `ipfs://` addresses onto the URI handling of EMF, the lifecycle of a resource that receives an identifier only after the first save, and the coexistence of file resources and content-addressed resources inside one resource set.

RQ2. Cross-resource references. How can references between resources be maintained and resolved when the identifier of every resource changes on modification? Sub-questions cover the available strategies, the update of a referencing resource after a change, the treatment of reference cycles, and the publication of a model that is split into many resources.

RQ3. Performance. What is the overhead of content-addressed persistence, and under which conditions is the approach practical for an MDE toolchain? Sub-questions cover the cost of a save, the cost of a load, the behaviour as the model grows, and the point where the overhead stops being acceptable for interactive work.

6 Method

The study follows design science, where a designed artifact answers the research questions and an evaluation of the artifact gives the evidence [10]. The case study part follows the guidelines of Runeson

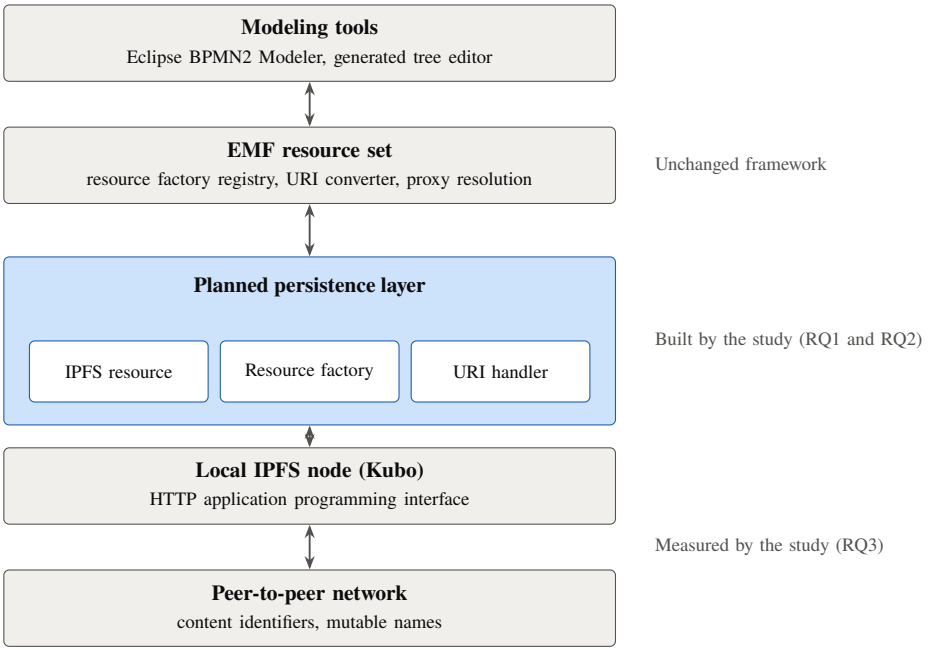


Figure 3: Planned place of the persistence layer. The framework above stays unchanged, the layer in the middle is the artifact of the study, and the node below gives the storage.

and Höst [18]. The work has four steps.

The first step analyses the problem. The analysis compares the assumptions of the EMF resource model with the properties of content addressing, and produces a list of the points where the two disagree.

The second step designs and builds the artifact. The artifact is a plugin that adds IPFS persistence to EMF, together with the strategies for references and the algorithm that updates referencing resources.

The third step validates the functions. Automated tests and example programs check save and load round trips, mixed resource sets, cross-resource references and proxy resolution.

The fourth step measures cost and scale. A benchmark on generated models compares a save through IPFS with a save to a local XMI file, and a case study on large reverse-engineered models tests the approach at a realistic size. Figure 4 shows the four steps with the output of each step and with the question that each step serves.

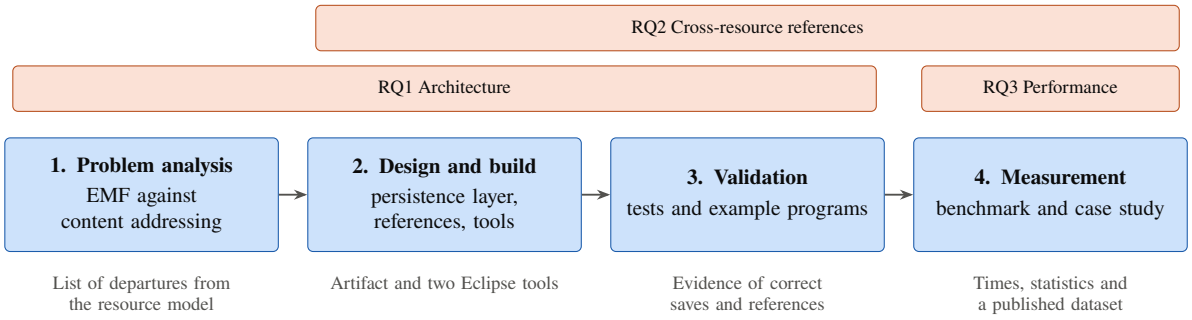


Figure 4: Research design. The four steps of the study, the output of each step, and the research question that each step serves.

7 Work plan

Package	Work	Months
WP1	Problem analysis and literature review, with a verified reference list	1
WP2	Persistence layer for `ipfs://` and `ipns://` (RQ1)	1 to 3
WP3	Reference strategies, the cascade update and the publication structure (RQ2)	2 to 4
WP4	Tool support in the Eclipse BPMN2 Modeler and in the generated tree editor	3 to 4
WP5	Evaluation: benchmark, functional tests and the case study (RQ3)	4 to 5
WP6	Writing, artifact release and submission	5 to 6

Figure 5 shows the schedule. Milestones follow the packages. M1 at month 1 is the problem analysis with the reference list. M2 at month 3 is a persistence layer that saves and loads a model through a local node. M3 at month 4 is a working update of referencing resources with tests, together with the tool support inside an editor. M4 at month 5 is the complete evaluation data. M5 at month 6 is the submitted manuscript with a public artifact.

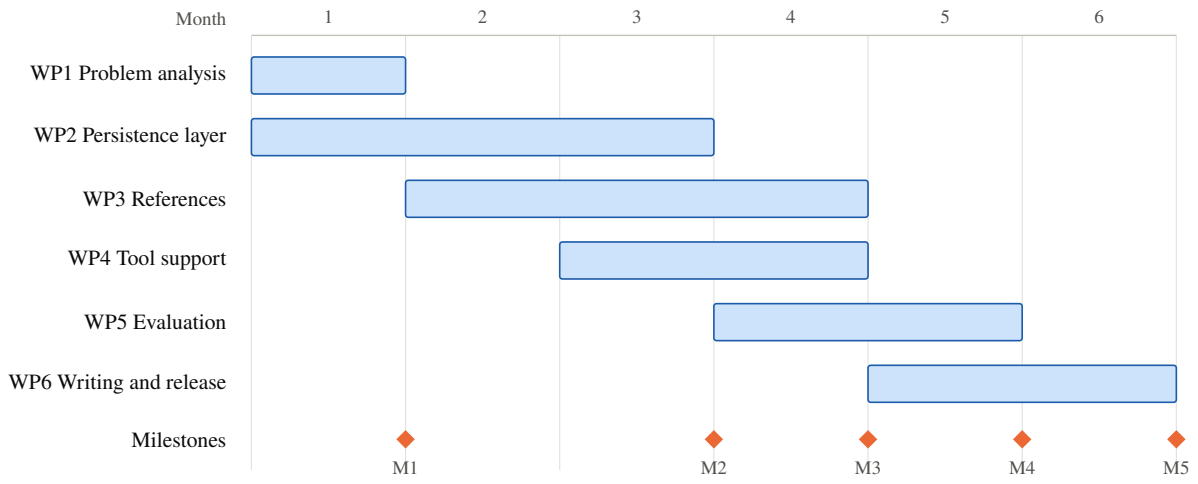


Figure 5: Schedule of the work packages over six months, with the five milestones.

8 Evaluation plan

The evaluation combines three methods.

Functional validation uses automated tests and example programs. The tests will cover a save and a load round trip, a resource set that holds both file resources and content-addressed resources, a reference across two resources, and the resolution of a proxy after a load in a fresh resource set.

A benchmark measures the cost. Generated models of growing size, from one hundred to several hundred thousand elements, will be saved to a local XMI file and to a local IPFS node inside the same iteration, and loaded back in a fresh resource set. Each size will run warm-up iterations followed by at least thirty measured iterations, and a paired statistical test will compare the two paths. The measurement will record the environment, because a busy machine and a node that announces new content to the network both distort the numbers.

A case study tests the approach at a realistic size. Reverse-engineered models of the Eclipse Java development tools, produced with MoDisco [3], give roughly one gigabyte of models across several

repositories. The case study will publish the models on IPFS, both as one merged model and as a model split into one resource per compilation unit, and will then fetch the published copies back. Byte-level comparison and EMF Compare will check that a published model returns unchanged.

9 Expected contributions and deliverables

The study will deliver four contributions. The first is an EMF resource implementation that saves models on IPFS and loads models by content identifier or by IPNS name. The second is a description of the strategies for references between content-addressed resources, together with an algorithm that updates referencing resources after a change. The third is IPFS support inside two Eclipse tools, so the approach reaches a user interface. The fourth is an evaluation with generated models and with large reverse-engineered models.

The deliverables are public repositories with the plugin and the tool integrations, the evaluation scripts and result files, an archived dataset with a digital object identifier (DOI), and a journal manuscript for SoSyM. An earlier conference paper will cover the reference question, which carries the main new contribution.

10 Risks and mitigation

Risk	Effect	Mitigation
The resource model of EMF resists a changing identifier	The persistence layer stays incomplete	Early proof of concept in WP2, with a placeholder address before the first save
A reference cycle cannot settle under immutable identifiers	Some models fall outside the approach	Detect cycles before publication, and report the limit honestly
The update of referencing resources becomes expensive on a large graph	The approach fails at scale	Measure on the case study model, and compare a scan with an index built from manifests
Measurements vary with the machine and with the node	The reported numbers mislead	Fixed heap, warm-up iterations, an idle machine, a node without network announcements, and a paired test
A published model becomes unavailable	The evaluation cannot be repeated	Pin the content on a node under project control, and archive the dataset under a DOI

11 Resources and budget

The core work needs one researcher, a development machine and a local IPFS node. The evaluation beyond a single machine needs a second machine on another network, because a claim about decentralized persistence needs more than one node. Each machine is reached through a tunnel that is bound to a domain name, so the application programming interface of a node stays closed to the open internet.

Category	Purpose	USD	IDR
Researcher	Honorarium, six months at IDR 2,000,000 per month	706	12,000,000
Assistant	Part time, six months at IDR 2,000,000 per month	706	12,000,000
Storage and pinning	Retention of the published dataset	250	4,250,000

Category	Purpose	USD	IDR
Domain names	Two names at Cloudflare for the tunnels, where the service itself costs nothing	22	374,000
Archiving	Deposit under a digital object identifier, free on Zenodo	0	0
Flight	Return flight to a regional conference in Japan or Korea	600	10,200,000
Accommodation	Four nights near the venue	360	6,120,000
Registration	One full conference registration	600	10,200,000
Meals and transport	Five days	250	4,250,000
Subtotal	Without the open access fee	3,494	59,394,000
Publication	Open access fee of the journal, optional	3,190	54,230,000
Total	With the open access fee	6,684	113,624,000

The amounts are estimates at an exchange rate of IDR 17,000 per USD, and the figures follow public provider prices of September 2026. The open access fee comes from the price list of the journal, where the current article processing charge is USD 3,190. The subscription route of the same journal carries no fee, so the last row stays optional. Personnel amounts follow institutional rates and need a check against the rules of the funding scheme.

References

[1] Benet, J.: IPFS, content addressed, versioned, peer-to-peer file system. arXiv:1407.3561 (2014). https://arxiv.org/abs/1407.3561

[2] Barmpis, K., Kolovos, D.: Hawk, towards a scalable model indexing architecture. In: BigMDE 2013, pp. 1 to 9. ACM (2013). https://doi.org/10.1145/2487766.2487769

[3] Brunelière, H., Cabot, J., Dupé, G., Madiot, F.: MoDisco, a model driven reverse engineering framework. Information and Software Technology 56(8), 1012 to 1032 (2014). https://doi.org/10.1016/j.infsof.2014.04.007

[4] Eclipse Foundation: CDO Model Repository. https://eclipse.dev/cdo/

[5] Chacon, S., Straub, B.: Pro Git, 2nd edition. Apress, Berkeley (2014). https://git-scm.com/book/en/v2

[6] Daniel, G., Sunyé, G., Benelallam, A., Tisi, M., Vernageau, Y., Gómez, A., Cabot, J.: NeoEMF, a multi-database model persistence framework for very large models. Science of Computer Programming 149, 9 to 14 (2017). https://doi.org/10.1016/j.scico.2017.08.002

[7] Di Cosmo, R., Zacchiroli, S.: Software Heritage, why and how to preserve software source code. In: iPRES 2017, pp. 1 to 10. https://hal.science/hal-01590958

[8] Dolstra, E., de Jonge, M., Visser, E.: Nix, a safe and policy-free system for software deployment. In: LISA 2004, pp. 79 to 92. USENIX, Berkeley (2004). https://www.usenix.org/legacy/publications/library/proceedings/lisa04/tech/dolstra.html

[9] Eclipse Foundation: EMF Compare. https://eclipse.dev/emfcompare/

[10] Hevner, A.R., March, S.T., Park, J., Ram, S.: Design science in information systems research. MIS Quarterly 28(1), 75 to 106 (2004). https://doi.org/10.2307/25148625

[11] Koegel, M., Helming, J.: EMFStore, a model repository for EMF models. In: ICSE 2010, vol. 2, pp. 307 to 308. ACM (2010). https://doi.org/10.1145/1810295.1810364

[12] Kolovos, D.S., Rose, L.M., Matragkas, N., Paige, R.F., et al.: A research roadmap towards achieving scalability in model driven engineering. In: BigMDE 2013, pp. 1 to 10. ACM (2013). https://doi.org/10.1145/2487766.2487768

[13] Lamb, C., Zacchiroli, S.: Reproducible builds, increasing the integrity of software supply chains. IEEE Software 39(2), 62 to 70 (2022). https://doi.org/10.1109/MS.2021.3073045

[14] npm Docs: package-lock.json. https://docs.npmjs.com/cli/v10/configuring-npm/package-lock-json

[15] Object Management Group: XML Metadata Interchange (XMI) specification, version 2.5.1 (2015). https://www.omg.org/spec/XMI/2.5.1

[16] Espinazo Pagán, J., Sánchez Cuadrado, J., García Molina, J.: A repository for scalable model management. Software and Systems Modeling 14(1), 219 to 239 (2015). https://doi.org/10.1007/s10270-013-0326-8

[17] Quinlan, S., Dorward, S.: Venti, a new approach to archival storage. In: FAST 2002, pp. 89 to 101. USENIX, Berkeley (2002). https://www.usenix.org/legacy/events/fast02/quinlan.html

[18] Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering 14(2), 131 to 164 (2009). https://doi.org/10.1007/s10664-008-9102-8

[19] Steinberg, D., Budinsky, F., Paternostro, M., Merks, E.: EMF, Eclipse Modeling Framework, 2nd edition. Addison-Wesley, Upper Saddle River (2008).

[20] Trautwein, D., Raman, A., Tyson, G., Castro, I., Scott, W., Schubotz, M., Gipp, B., Psaras, Y.: Design and evaluation of IPFS, a storage layer for the decentralized web. In: ACM SIGCOMM 2022, pp. 739 to 752. ACM (2022). https://doi.org/10.1145/3544216.3544232